

1. Write a program that subtracts two numbers so that the result is negative. Then perform another subtraction that gives zero in output and check the SF and ZF flags.

Code:

```
.model small

.stack 100h

.data

    num1 db 0x05
    num2 db 0x08
    num3 db 0x06
    num4 db 0x06


.code

main proc

    mov ax, @data
    mov ds, ax


    ;first subtraction

    mov al, num1
    sub al, num2

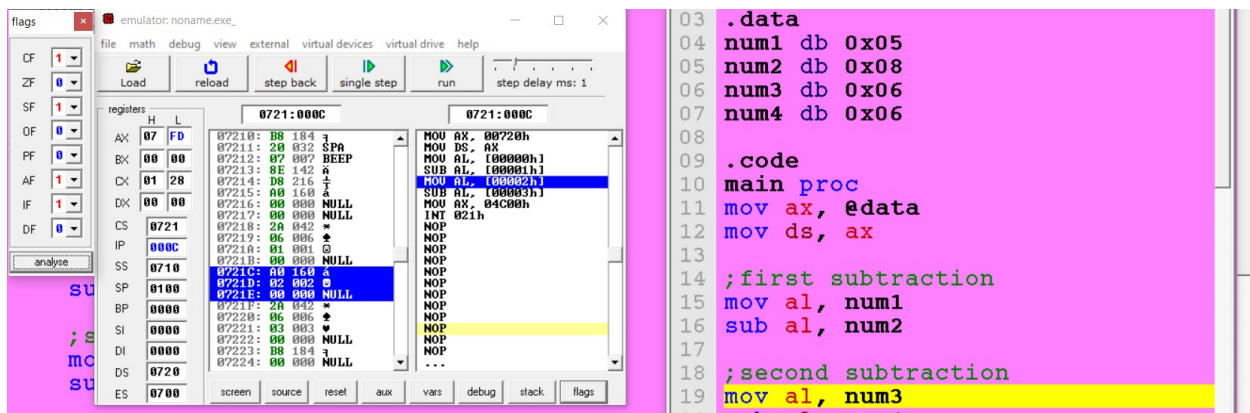

    ;second subtraction

    mov al, num3
    sub al, num4
```

main endp

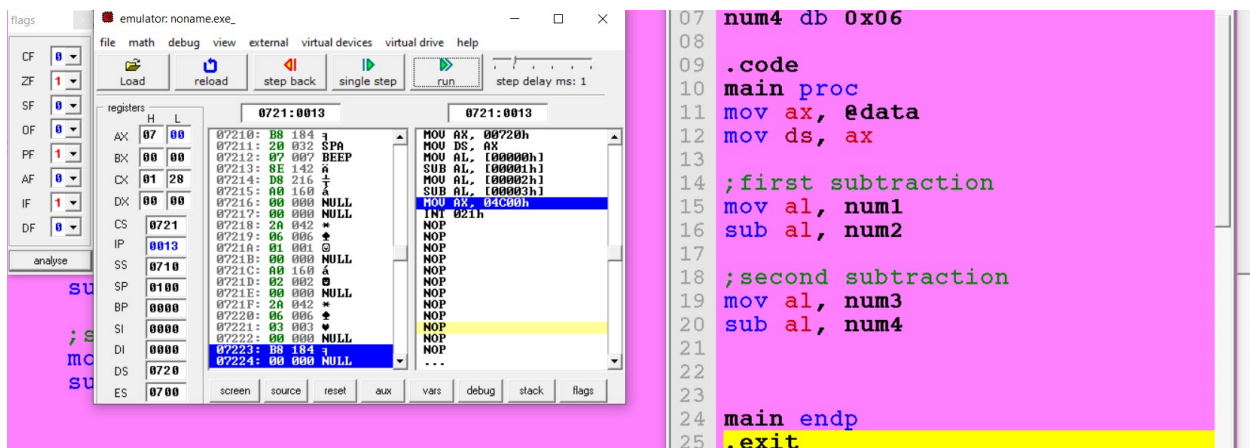
.exit

First subtraction:



```
03 .data
04 num1 db 0x05
05 num2 db 0x08
06 num3 db 0x06
07 num4 db 0x06
08
09 .code
10 main proc
11 mov ax, @data
12 mov ds, ax
13
14 ;first subtraction
15 mov al, num1
16 sub al, num2
17
18 ;second subtraction
19 mov al, num3
```

Second Subtraction:



```
07 num4 db 0x06
08
09 .code
10 main proc
11 mov ax, @data
12 mov ds, ax
13
14 ;first subtraction
15 mov al, num1
16 sub al, num2
17
18 ;second subtraction
19 mov al, num3
20 sub al, num4
21
22
23
24 main endp
25 .exit
```

2. Write a program that stores the given two 32-bit numbers into the current data segment at offset: 0x1000 and 0x1008, respectively. The program then calculates the sum of these numbers and stores it at the offset: 0x1010.
Numbers: 0x1F540398, 0xC0A1F02E.

Code:

```
.model small
```

```
.stack 100h
```

```
;2.      Write a program that stores the given two 32-bit numbers into the
```

```
;current data segment at offset: 0x1000 and 0x1008, respectively.
```

```
;The program then calculates the sum of these numbers and stores it at
```

```
;the offset: 0x1010.
```

```
;Numbers: 0x1F540398, 0xC0A1F02E.
```

```
.data
```

```
num1 dw 0x0398 ;lower word
```

```
num2 dw 0x1f54 ;upper word
```

```
num3 dw 0xf02e ;lower word
```

```
num4 dw 0xc0a1 ;upper word
```

```
.code
```

```
main proc
```

```
    mov ax, @data
```

```
    mov ds, ax
```

```
mov bx, 0x1000 ;offset
```

```
;store in memory
```

```
mov si, 0
```

```
mov ax, num2
```

```
mov [bx + si], ax
```

```
add si, 2
```

```
mov ax, num1
```

```
mov [bx + si], ax
```

```
add si, 6
```

```
mov ax, num4
```

```
mov [bx + si], ax
```

```
add si, 2
```

```
mov ax, num3
```

```
mov [bx + si], ax
```

```
;load lower words from memory and sum
```

```
mov si, 2
```

```
mov ax, [bx + si]
```

```
add si, 8
```

```
mov dx, [bx + si]
```

```

add ax, dx
mov [bx + 0xc], ax

;load lower words from memory and sum
mov si, 0
mov ax, [bx + si]
add si, 8
mov dx, [bx + si]
adc ax, dx
mov [bx + 0xa], ax

main endp
.exit

```

Memory:

Random Access Memory

0720:1000

update

☒ table

☐ list

0720:1000	98	03	54	1F	00	00	00	00-2E	F0	A1	C0	00	00	00	00	ÿwT.....≡íL...
0720:1010	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1020	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1030	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1040	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1050	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1060	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	
0720:1070	00	00	00	00	00	00	00	00-00	00	00	00	00	00	00	00	

Output stored in memory:

Random Access Memory

0720:1000

update

☒ table

☐ list

0720:1000	54	1F	98	03	00	00	00	00	00	A1	C0	F5	DF	C6	F3	00	00	Tvü.....iJ■k.
0720:1010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
0720:1070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

3. Write code that set both PF and ZF.

Code:

```
.model small
```

```
.stack 100h
```

```
;3. Write code that set both PF and ZF.
```

```
.data
```

```
.code
```

```
main proc
```

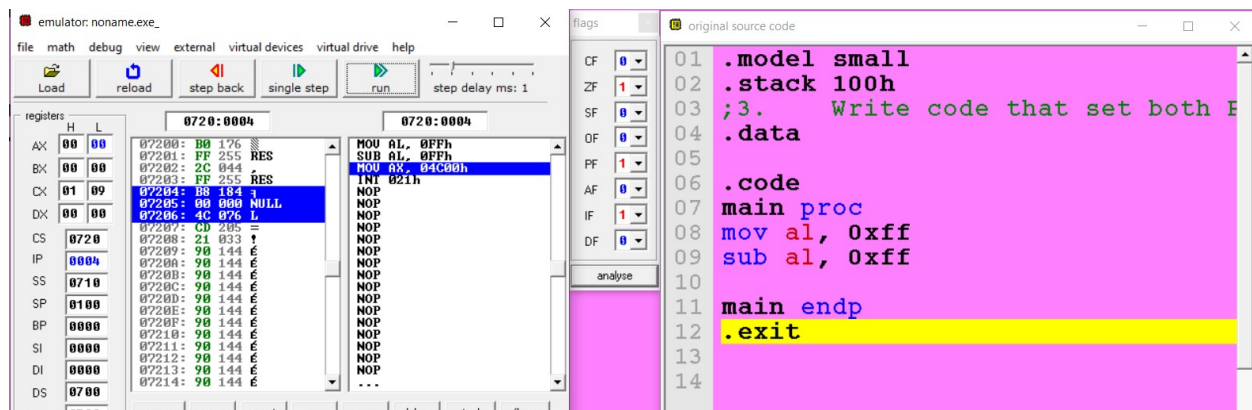
```
mov al, 0xff
```

```
sub al, 0xff
```

```
main endp
```

```
.exit
```

Flags:



4. Write a program that sub two 16-bit numbers stored in memory at offset x2000h and checks if ZF is set. If ZF = 1, store 1 at memory location 4000h; otherwise, store at 0x1000h.(use jmp)

Code:

```
.model small
```

```
.stack 100h
```

```
;4. Write program that sub two 16-bit numbers stored in memory at  
offset x2000h
```

```
;and checks if ZF is set. If ZF = 1, store 1 at memory location 4000h;
```

```
;otherwise, store at 0x1000h.(use jmp)
```

```
.data
```

```
num1 dw 0x1234
```

```
num2 dw 0x1234
```

```
.code
```

```
main proc
```

```
mov ax, @data
```

```
mov ds, ax
```

```
mov bx, 0x2000 ;offset
```

```
;store in memory
```

```
mov ax, num1
```

```
mov [bx], ax
```

```
mov ax, num2
```

```
mov [bx + 2], ax
```

```
;load from memory and subtract
```

```
mov ax, [bx]
```

```
sub ax, [bx + 2]
```

```
jz mov_to_0x4000
```

```
mov [bx - 0x1000], ax
```

```
jmp end
```

```
mov_to_0x4000:
```

```
mov [bx + 0x2000], ax
```

```
main endp
```


end:

.exit

ZF is set:

The image shows a debugger interface with two main windows. The top-left window, titled 'Random Access Memory', displays a memory dump starting at address 0720:4000. The dump shows several lines of memory addresses and their corresponding hex values, mostly 00. The top-right window, titled 'original source code', shows assembly code. Line 32, 'mov [bx + 0x2000], ax', is highlighted in yellow. The bottom window shows the CPU registers and the instruction stream. The instruction stream shows the current instruction being executed, which is 'mov [bx + 0x2000], ax'. The registers window shows the state of the CPU registers, with AX containing 0000 and BX containing 0000. The flags window shows the status of the ZF flag, which is set to 1.

Random Access Memory

0720:4000 update table list

0720:4000 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4010 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4020 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4030 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4040 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4050 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4060 00 00 00 00 00 00 00-00 00 00 00 00 00 00
0720:4070 00 00 00 00 00 00 00-00 00 00 00 00 00 00

emulator: noname.exe

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 1

registers H L

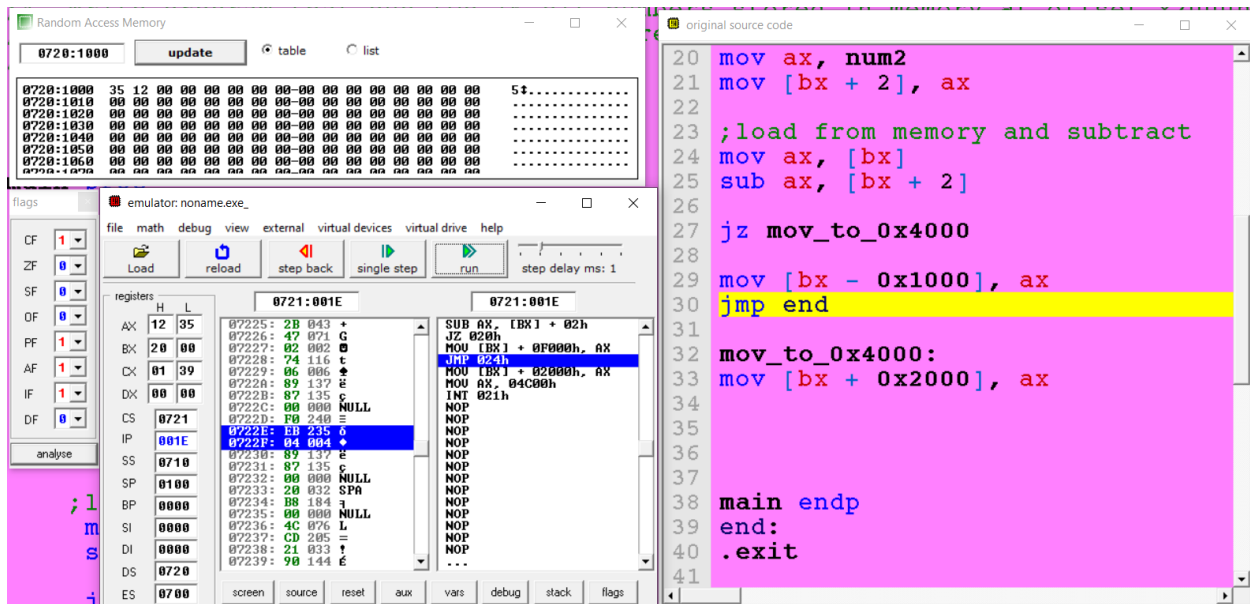
AX 00 00 0721:0020 0721:0020
BX 20 00
CX 01 39
DX 00 00
ES 0720
EB 235 6
ED 004 4
EE 89 137 E
EF 87 135 G
07222: 00 000 NULL
07223: 20 032 SPB
07224: B8 184 3
07225: 4C 076 L
07226: CD 205 =

MOV AX, 00720h
MOV DS, AX
MOV BX, 02000h
MOV AX, [00000h]
MOV [BX], AX
MOV AX, [00002h]
MOV [BX] + 02h, AX
SUB AX, [BX] + 02h
JZ 020h
MOV [BX] + 0F000h, AX
JMP 024h
MOV [BX] + 02000h, AX
MOV AX, 04C00h
INT 021h
NOP
NOP
NOP
NOP
...

original source code

```
20 mov ax, num2
21 mov [bx + 2], ax
22
23 ;load from memory and subtract
24 mov ax, [bx]
25 sub ax, [bx + 2]
26
27 jz mov_to_0x4000
28
29 mov [bx - 0x1000], ax
30 jmp end
31
32 mov_to_0x4000:
33 mov [bx + 0x2000], ax
34
35
36
37
38 main endp
39 end:
40 .exit
41
```

ZF is not set:



5. Write a program to implement the following equation.

$$X = \sim 0xFF12 \wedge \{0xABFF \& (0x2113 \mid 0x2340)\}$$

~ Invert all bits

^ Bitwise XOR

& Bitwise AND

| Bitwise OR

Code:

```
.model small
```

```
.stack 100h
```

```
.data
```

```
num1 dw 0xFF12
num2 dw 0xABFF
num3 dw 0x2113
num4 dw 0x2340
x dw 0x0000

.code
main proc
    mov ax, @data
    mov ds, ax

    mov ax, num3
    mov dx, num4
    or ax, dx

    mov dx, num2
    and ax, dx

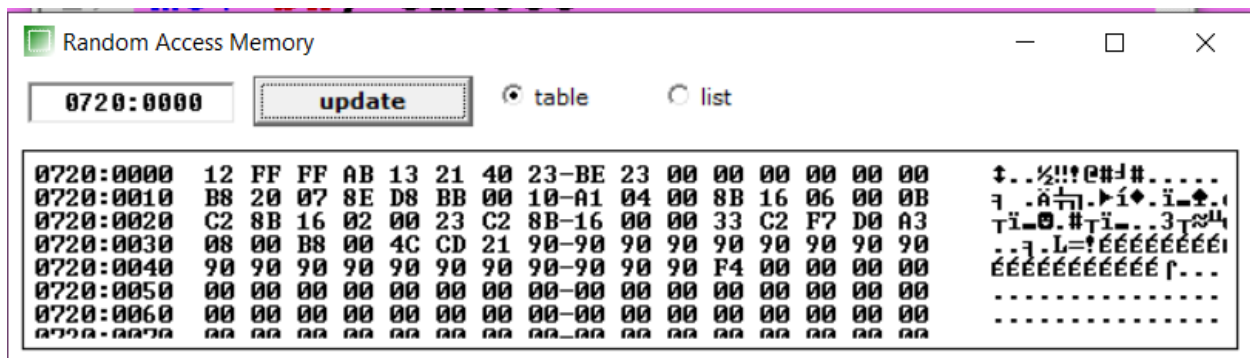
    mov dx, num1
    xor ax, dx
    not ax

    mov x, ax

main endp
```

```
.exit
```

Stored in memory:



6. Write a program that stores a 32-bit number and a 16-bit number into the current data segment at offsets 0x1000 and 0x1008, respectively. The program then subs these two numbers. 0xC0A1F02E, 0x45FA

Code:

```
.model small
```

```
.stack 100h
```

Write a program that stores a 32-bit number and a 16-bit number into the

```
;current data segment at offsets 0x1000 and 0x1008, respectively.
```

;The program then subs these two numbers. 0xC0A1F02E, 0x45FA

.data

num1 dw 0xF02E ;lower word

num2 dw 0xC0A1 ;upper word

num3 dw 0x45FA ;second number

lresult dw 0x0000

uresult dw 0x0000

.code

main proc

mov ax, @data

mov ds, ax

mov bx, 0x1000 ;offset

;store in memory

mov si, 0

mov ax, num2

mov [bx + si], ax

add si, 2

mov ax, num1

mov [bx + si], ax

add si, 6

```
mov ax, num3
```

```
mov [bx + si], ax
```

```
;load from memory and subtract
```

```
mov si, 2
```

```
mov ax, [bx + si]
```

```
add si, 6
```

```
mov dx, [bx + si]
```

```
sub ax, dx
```

```
;store in result variable
```

```
mov dx, [bx]
```

```
mov uresult, dx ;stored at 0720:0000 memory(global variables)
```

```
mov lresult, ax ;stored at 0720:0000 memory(global variables)
```

```
main endp
```

```
.exit
```

Stored in memory:

7. Write an assembly program to store **5 numbers** in an array and then add them.

Code:

```
.model small
```

```
.stack 100h
```

;7. Write an assembly program to store 5 numbers in an array and then add them.

```
.data
```

```
array dw 0xaaaa, 0xbbbb, 0xcccc, 0xdddd, 0xeeee
```

```
sum dw 0x0000
```

```
.code
```

```
main proc
```

```
    mov ax, @data
```

```
    mov ds, ax
```

```
    mov cx, 0x0005
```

```
    mov si, offset array
```

```
    mov ax, 0
```

```
loop:
```

```
    add ax, [si]
```

```
    add si, 2
```

```
    loop loop
```

```
    mov sum, ax
```


main endp

.exit

Saved in memory:

The image shows a debugger interface with two main windows. The left window, titled 'Random Access Memory', displays a memory dump starting at address 0720:0000. The right window, titled 'original source code', shows the assembly code for a program. The code includes a data section with an array of three words (0xaaaa, 0xbbbb, 0xcccc) and a code section with a loop that iterates over the array, adding the values to a register (AX). The code ends with a main endp and .exit instruction.

Random Access Memory

Address	Hex	ASCII
0720:0000	AA AA BB CC CC DD DD EE EE FC FF 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0010	B8 20 07 8E D8 B9 05 00 00 00 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0020	83 C6 02 E2 F9 A3 00 00 00 00 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0030	90 90 90 90 90 90 90 90 90 90 90 90 90 90	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0040	90 F4 00 00 00 00 00 00 00 00 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
0720:0060	00 00 00 00 00 00 00 00 00 00 00 00 00 00	AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

emulator: task_7.exe

registers

Register	Value
AX	FF FC
BX	00 00
CX	00 00
DX	00 00
SI	0000
DI	0000
DS	0720
ES	0700

original source code

```
04 .data
05 array dw 0xaaaa, 0xbbbb, 0xcccc,
06 sum dw 0x0000
07 .code
08 main proc
09 mov ax, @data
10 mov ds, ax
11
12 mov cx, 0x0005
13 mov si, offset array
14 mov ax, 0
15
16 loop:
17 add ax, [si]
18 add si, 2
19 loop loop
20
21 mov sum, ax
22
23 main endp
24 .exit
25
```

8. Take a variable in .data section, and calculate its table. (use loops).

Code:

```
.model small
```

```
.stack 100h
```

```
;8. Take a variable in .data section, and calculate its table. (use loops).
```

```
.data
```

```
    num dw 0x0005
```

```
.code
```

```
main proc
```

```
    mov ax, @data
```

```
    mov ds, ax
```

```
    mov ax, num
```

```
    mov cx, 0x0009
```

```
    ;loop runs 9 times
```

```
loop:
```

```
    add ax, 0x0005
```

loop loop

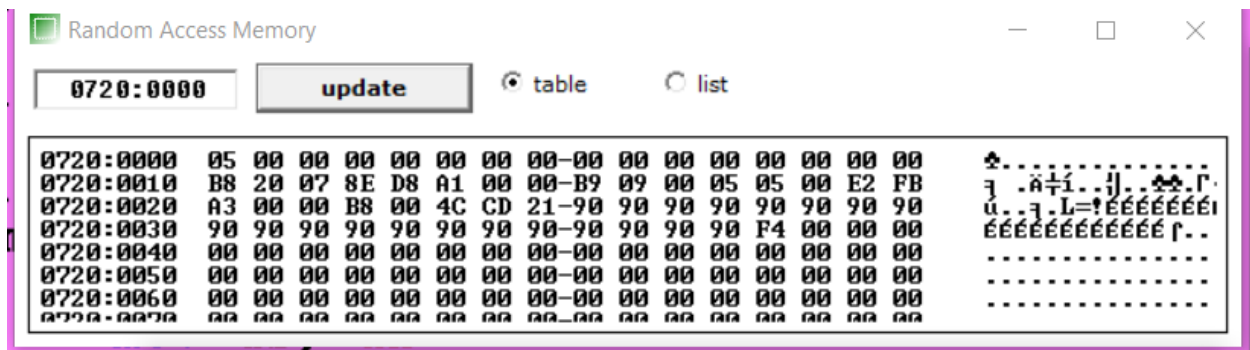
;save in memory variable

mov num, ax

main endp

.exit

Number 0x0005:



Address	Hex	ASCII
0720:0000	05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0720:0010	B8 20 07 8E D8 A1 00 00 B9 09 00 05 05 00 E2 FB	
0720:0020	A3 00 00 B8 00 4C CD 21 90 90 90 90 90 90 90 90	
0720:0030	90 90 90 90 90 90 90 90 90 90 90 90 F4 00 00 00	
0720:0040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0720:0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0720:0060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
0720:0070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Last value after loop running to print table:

